Processor design

We will implement a processor on an FPGA

Soft-Core Processor

A **soft processor** is a CPU described in **HDL (Verilog/VHDL)** and implemented using FPGA logic resources (LUTs, flip-flops, BRAM, etc.).

- Processor is **not physically fixed**
- It is **configured/programmed onto FPGA fabric**
- Fully customizable (ISA, pipeline, accelerators, etc.)

Phase 1: Motivation

- dedicated hardware vs processor
- why programmability is important
- why same hardware should support many programs

Dedicated hardware vs. General-purpose processor

$$y = a*b + c$$

Dedicated hardware implementation

```
a ─┐
   ├► MUL ─┐
b ─┘       ├► ADD ──► y
c ─────────┘
```

Computation happens **in 1 clock cycle (or few cycles)**

Processor-based implementation

‹/› asm

```asm
LOAD R1, a
LOAD R2, b
MUL  R3, R1, R2
LOAD R4, c
ADD  R5, R3, R4
STORE R5, y
```

Multiple instructions → multiple cycles

Dedicated hardware gives speed

Processor gives flexibility

Dedicated hardware vs. General-purpose processor

- Dedicated hardware implementations provide high performance for specific computations by exploiting parallelism and fixed datapaths.
- In contrast, general-purpose processors offer flexibility by executing a sequence of instructions over a shared datapath.
- While processors incur additional overhead due to instruction fetch and control, they enable reprogrammability and support a wide range of applications.
- Modern computing systems often combine both approaches to achieve a balance between performance and flexibility.

A general-purpose RISC CPU

A **general-purpose RISC/CISC CPU** is designed to:

- execute *many types of instructions*
- be flexible and programmable
- handle control-heavy tasks (branching, loops, memory)

A general-purpose RISC CPU

- But it is **not optimized for repetitive arithmetic workloads**.

$$y = \sum x_i \cdot h_i$$

1. Problem with CPU-only execution

Take your FIR or dot product example:

$$y = \sum x_i \cdot h_i$$

On CPU, each term requires:

- LOAD x_i
- LOAD h_i
- MULTIPLY
- ADD
- LOOP CONTROL (branch, increment)

So one simple MAC operation becomes multiple instructions + multiple cycles

2. Nature of modern workloads

Many important applications are dominated by:

- **DSP** → FIR, FFT, filtering
- **ML / AI** → matrix multiplication, convolution
- **Signal processing** → correlation, feature extraction

These workloads are:

- **highly repetitive**
- **arithmetic-heavy**
- **parallelizable**

3. CPU inefficiency for such tasks

CPU executes them as:

- sequential instructions
- repeated memory access
- repeated ALU reuse

So:

- **high instruction count**
- **high cycle count**
- **low throughput**

Dedicated hardware block

- A specialized datapath designed to perform a specific operation directly in hardware.

Example:

- MAC unit → does $A \times B + ACC$
- Convolution engine → does sliding-window MACs
- FFT unit → does butterfly operations

Instead of breaking into instructions, it performs the **entire operation as a hardware function**.

Dedicated hardware block → “accelerator”

- Because it **accelerates execution** compared to CPU-only execution.

CPU = general worker

Can do everything, but slowly for repetitive tasks

Accelerator = specialized machine

Built only for one task, but does it extremely fast

Accelerators

- Dedicated hardware blocks are specialized computational units designed to execute specific arithmetic or data-processing operations more efficiently than a general-purpose processor.
- These blocks exploit parallelism, data locality, and customized datapaths to reduce instruction count, execution cycles, and energy consumption. Such units are referred to as accelerators because they significantly speed up the execution of compute-intensive workloads compared to software execution on a CPU.

What is the coprocessor approach?

A specialized hardware unit that works alongside a general-purpose CPU to accelerate specific types of computations.

So the system becomes:

$$\text{CPU (control + general tasks)} + \text{Coprocessor (specialized compute)}$$

Modern systems combine both:

Processor + Hardware accelerators

- CPU + GPU
- CPU + DSP
- CPU + MAC unit
- CPU + AI accelerator

Core idea of coprocessor approach

CPU handles

- control flow (loops, branching)
- memory management
- scheduling
- orchestration

Coprocessor handles

- heavy computation
- repetitive arithmetic
- parallel operations

Coprocessor in modern systems

1. GPU (Graphics Processing Unit)

The most famous coprocessor.

- CPU → sends rendering or compute tasks
- GPU → executes thousands of parallel operations

Used for:

- graphics rendering
- deep learning (matrix multiply, convolution)

2. AI / ML accelerators (TPU, NPU)

Specialized for neural networks.

- optimized for:
  - MAC operations
  - tensor operations
  - low precision arithmetic (INT8, FP16)

Examples:

- Google TPU
- Apple Neural Engine
- Edge NPUs in mobile devices

A high-performance MCU with integrated AI/vision accelerators

STM32N6 = Heterogeneous SoC

$$\text{CPU} + \text{AI accelerator}$$

```
CPU only
   ↓
CPU + MAC (coprocessor-based)
   ↓
CPU + CNN accelerator
   ↓
CPU + multiple accelerators
   ↓
Heterogeneous SoC
```

```
y = a + b
y = a - b
y = a * b
y = a * b + c
y = (a + b) - c
y = a * 3 + 2
```

```
Summary:
Program                          Output     Avg Time (s)
-------------------------------------------------------------
y = a + b                        12         1.256059589994e-07
y = a - b                        5          1.138131819991e-07
y = a * b                        18         1.193566119997e-07
y = a * b + c                    11         1.576415319997e-07
y = (a + b) - c                  7          1.486810780002e-07
y = a * 3 + 2                    14         1.303173890000e-07
```

System Manufacturer	HP
System Model	HP Laptop 14s-fq1xxx
System Type	x64-based PC
System SKU	4U582PA#ACJ
Processor	AMD Ryzen 7 5700U with Radeon Graphics, 1801 Mhz, 8 Core(s),

CISC processor

x86-64 architecture

CISC = Complex Instruction Set Computer

- Variable-length instructions
- Complex instructions (e.g., memory + ALU in one instruction)
- Many addressing modes

Project objective

- We will design and implement a simple general-purpose multi-cycle RISC processor on FPGA. The processor will support a basic instruction set for arithmetic, data movement, and control flow. Its correctness will be validated by executing multiple simple programs derived from high-level code examples such as addition, subtraction, multiplication, and compound expressions like a*b+c.

Our objective

- A multi-cycle RISC processor will be developed.
- The processor supports a basic load/store instruction set with arithmetic, immediate, memory, and branch operations.
- Instruction execution is divided into fetch, decode, execute, memory access, and write-back stages controlled by a finite state machine.
- By reusing common datapath components across multiple cycles, the design reduces hardware complexity while clearly exposing the internal execution flow of each instruction.
- The processor is capable of executing a variety of high-level arithmetic expressions such as a+b, a-b, a*b, and a*b+c using the same hardware through different instruction sequences.

Phase 2: ISA design

Define a small base instruction set

Phase 3: Instruction format

Define encoding of instructions

Phase 4: Multi-cycle execution model

Explain fetch, decode, execute, memory, write-back

Phase 5: Datapath design

Design hardware blocks and temporary registers

Phase 6: Control FSM

Define state transitions for each instruction

Phase 7: Verilog implementation

Implement module by module

Phase 8: FPGA deployment

Download to board and observe output

Phase 9: Validation with HLL examples

Show multiple high-level programs running correctly

ISA

Data movement

- `LI Rd, imm`
- `LOAD Rd, addr`
- `STORE Rs, addr`
- `MOV Rd, Rs`

Arithmetic

- `ADD Rd, Rs1, Rs2`
- `SUB Rd, Rs1, Rs2`
- `MUL Rd, Rs1, Rs2`

Control

- `JMP addr`
- `BEQ Rs1, Rs2, addr`
- `BNE Rs1, Rs2, addr`
- `HALT`

- 8-bit data width
- 8 general-purpose registers
- 16-bit fixed instruction format
- separate instruction memory and data memory
- multi-cycle FSM control
- single active instruction at a time

RISC = Reduced Instruction Set Computer

Core ideas:

- Simple instructions
- Fixed instruction format
- **Load/Store architecture** (only LOAD/STORE access memory)
- Arithmetic happens only between registers

Program 1

HLL

```c
y = a + b;
```

Expected assembly idea

```asm
LOAD R1, 0
LOAD R2, 1
ADD  R3, R1, R2
STORE R3, 3
HALT
```

Program 4

HLL

```c
y = a * b + c;
```

Expected assembly

```asm
LOAD R1, 0
LOAD R2, 1
MUL  R3, R1, R2
LOAD R4, 2
ADD  R5, R3, R4
STORE R5, 3
HALT
```

- Program Counter (`PC`)
- Instruction Register (`IR`)
- Register File
- ALU
- Multiplier
- Data Memory
- Instruction Memory
- Temporary registers:

Define the tiny processor

Program

1. `LOAD R1, 10`
2. `LOAD R2, 11`
3. `ADD R3, R1, R2`
4. `SUB R4, R1, R2`
5. `MUL R5, R3, R2`
6. `STORE R5, 20`
7. `CMP R3, R4`
8. `HALT`

A simple fixed-width 16-bit RISC processor that supports:

- `ADD`
- `SUB`
- `MUL`
- `CMP`
- `LOAD`
- `STORE`
- `MOV`
- `HALT`

Blocks to introduce

- PC
- Instruction memory
- Instruction register
- Decoder / control unit
- Register file
- ALU
- Multiplier
- Data memory
- Flags register
- Write-back path

Problem statement

- Design a custom 16-bit fixed-width ISA supporting arithmetic, comparison, and load/store memory operations.
- Implement a multi-module RISC processor datapath in Verilog HDL: PC, instruction memory, register file, ALU, multiplier, data memory, and control unit.
- Design a synchronous control unit (decoder) that generates all datapath control signals from the 4-bit opcode.
- Execute the mandatory test program correctly and verify results against expected register and memory values.
- Synthesise, implement, and demonstrate the processor on the Boolean Board using switches, LEDs, and/or 7-segment display.
- Produce simulation waveforms showing correct fetch-decode-execute behaviour for each instruction in the test program.
- Document the design with a block diagram, ISA encoding table, datapath description, and implementation report.

Processor Specifications

Parameter	Specification
Data width	8-bit (all registers, ALU operands, and memory data are 8-bit)
Instruction width	16-bit fixed width (all instructions occupy exactly 16 bits)
General-purpose registers	8 registers — R0 through R7, each 8-bit wide
Program Counter (PC)	Increments by 1 after each instruction; no branch/jump support
Instruction memory	ROM / BRAM — preloaded with the program; addressed by PC
Data memory	RAM / BRAM — 512 locations, each 8-bit wide (9-bit address)
Execution model	Sequential, non-pipelined — one instruction completes before the next is fetched
Supported operations	NOP, ADD, SUB, MUL, CMP, MOV, LOAD, STORE, HALT
Comparison flags	Z (zero/equal), G (greater), L (less) — updated by CMP instruction only

PC	Instruction	16-bit Encoding	Operation
0	LOAD R1, 10	0110_001_000001010	R1 <= MEM[10] = 6
1	LOAD R2, 11	0110_010_000001011	R2 <= MEM[11] = 4
2	ADD R3, R1, R2	0001_011_001_010_000	R3 <= R1 + R2 = 10
3	SUB R4, R1, R2	0010_100_001_010_000	R4 <= R1 - R2 = 2
4	MUL R5, R3, R2	0011_101_011_010_000	R5 <= R3 x R2 = 40
5	STORE R5, 20	0111_101_000010100	MEM[20] <= R5 = 40
6	CMP R3, R4	0100_000_011_100_000	R3>R4 => G=1, Z=0, L=0

R-Type — Arithmetic, Logic, and Comparison Instructions

[15:12] opcode	[11:9] rd	[8:6] rs1	[5:3] rs2	[2:0] unused
4 bits	3 bits — destination register	3 bits — source register 1	3 bits — source register 2	3 bits — set to 000

Used by: ADD, SUB, MUL, CMP, MOV, NOP.

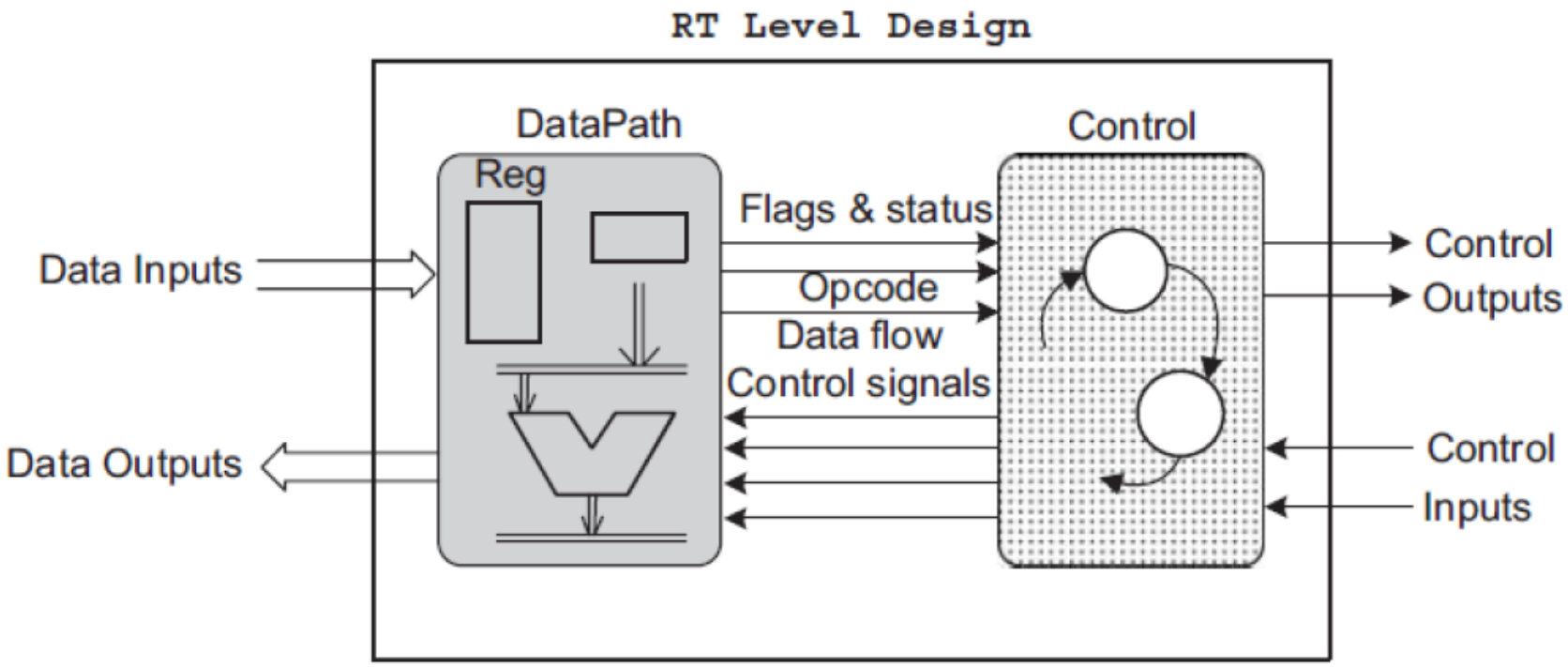
M-Type — Memory Instructions

[15:12] opcode	[11:9] rd / rs	[8:0] address
4 bits	3 bits — destination (LOAD) or source (STORE)	9 bits — data memory address (range 0 to 511)

Used by: LOAD, STORE.

Controller/datapath partitioning

The first step in an RT level design is the partitioning of the design into a data part and a control part. The data part consists of data components and the bussing structure of the design and the control part is usually a state machine generating control signals that control the flow of data in the data part.



risc_processor — xc7s50csga324-1

SLICE LOGIC

Slice LUTs	LUT as logic	LUT as dist. RAM	Slices used
198	166	32	58
0.61% of 32,600	0.51% · combinational	0.33% · data memory	0.71% of 8,150

RESOURCE UTILISATION — BAR VIEW

Slice LUTs	0.61%
Slice Registers	0.11%
Slices	0.71%
Block RAM	0.00%
DSP48E1	0.00%
IOBs	7.62%
BUFGCTRL	3.13%

IO USAGE

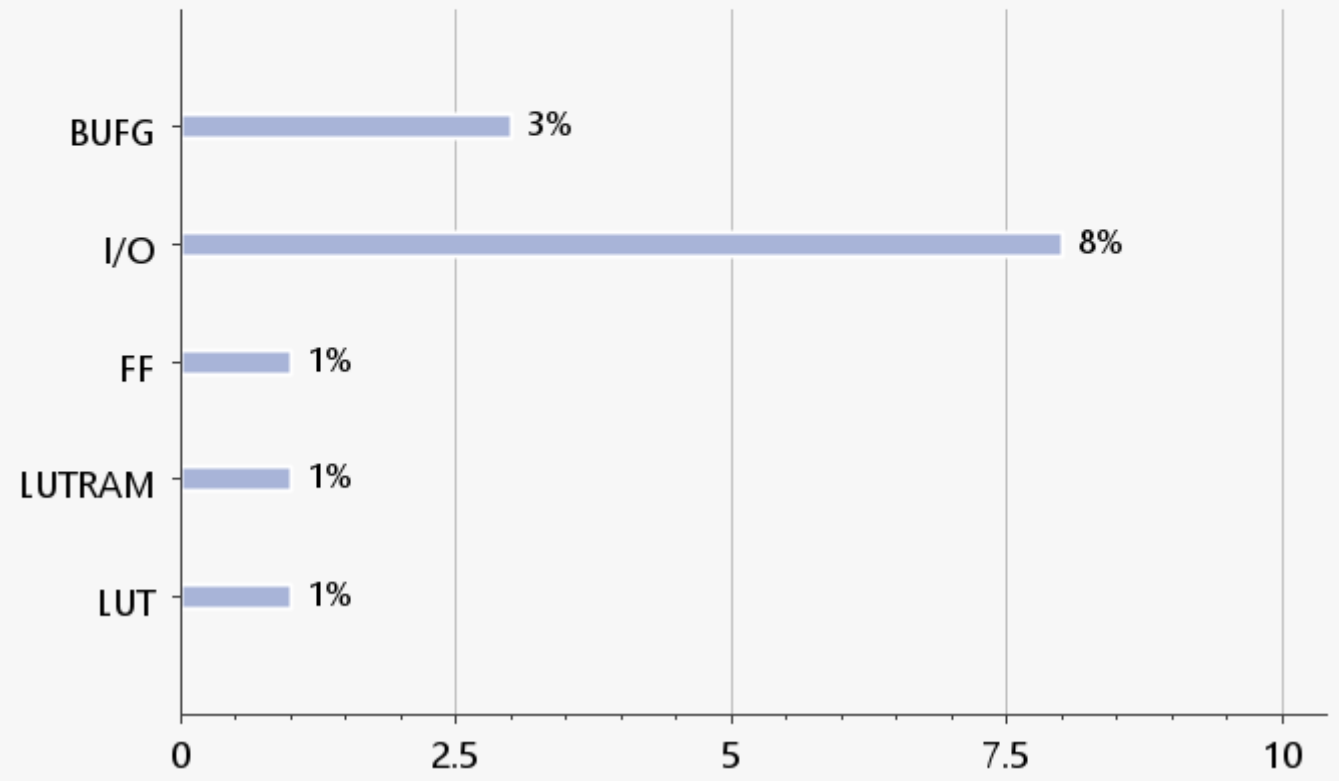
Bonded IOBs	IBUFs (inputs)	OBUFs (outputs)
16	5	11
7.62% of 210	clk, rst, SW[2:0]	LED[7:0] + flag_Z/G/L

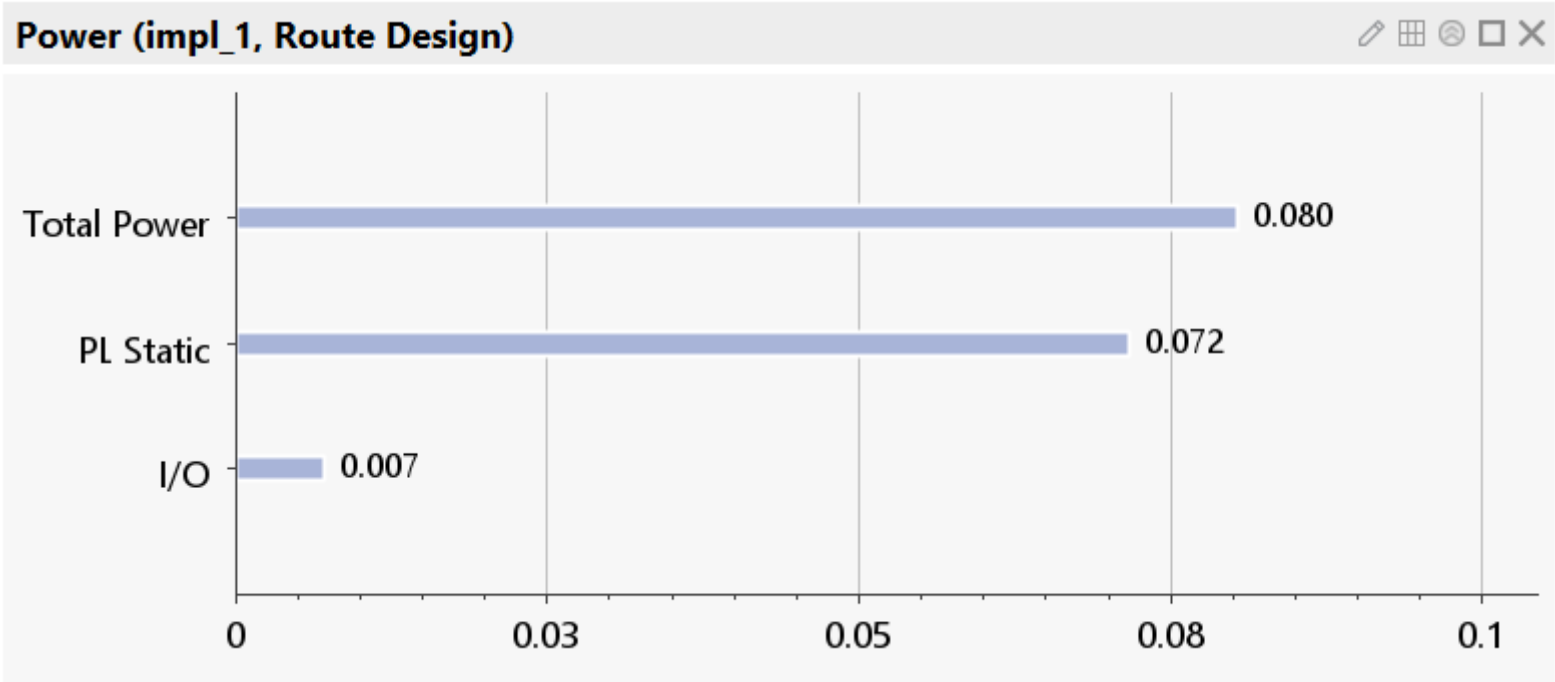
Clock period	WNS	Critical path	Fmax
10.0 ns	+1.062 ns	8.938 ns	111.9 MHz
100 MHz — Boolean Board	positive = timing met	= 10.0 − 1.062	= 1000 ÷ 8.938

Timing (impl_1, Route Design)

Report	WNS	WHS
impl_1, Timing Summary - Route Design	1.062	0.138

Utilization (impl_1, Place Design)




Power (impl_1, Route Design)
Total Power
PL Static
I/O
0.080
0.072
0.007
0
0.03
0.05
0.08
0.1

Verilog Design and Verification of Core Modules of a Simple 8-bit RISC Processor

The objective of this assignment is to design, code, and verify the core hardware modules of a simple 8-bit RISC processor using Verilog HDL. You will implement selected modules independently, test them using testbenches, and later use small encoded instruction programs to verify processor behaviour on the FPGA.

Processor Specification

The target processor has the following features:

- **8-bit datapath**
- **16-bit fixed-width instruction**
- **8 general-purpose registers**: `R0` to `R7`
- Separate **instruction memory** and **data memory**
- Dedicated **ALU**
- Dedicated **Multiplier module**
- Data memory supports **LOAD** and **STORE**

Opcode Table

Instruction	Binary	Hex
NOP	0000	0
ADD	0001	1
SUB	0010	2
MUL	0011	3
CMP	0100	4
MOV	0101	5
LOAD	0110	6
STORE	0111	7
HALT	1111	F

R-type Format:

[15:12] opcode | [11:9] rd | [8:6] rs1 | [5:3] rs2 | [2:0] unused

M-type Format:

[15:12] opcode | [11:9] reg | [8:0] memory address

Find out Encoded program

Assembly Program 1

```
LOAD  R1, 10
LOAD  R2, 11
ADD   R3, R1, R2
SUB   R4, R1, R2
MUL   R5, R3, R2
STORE R5, 20
CMP   R0, R3, R4
HALT
```

`program1.mem`

Assembly Program 2

```
LOAD  R1, 10
LOAD  R2, 11
ADD   R3, R1, R2
MUL   R4, R1, R2
STORE R4, 21
HALT
```

`program2.mem`

program1.mem

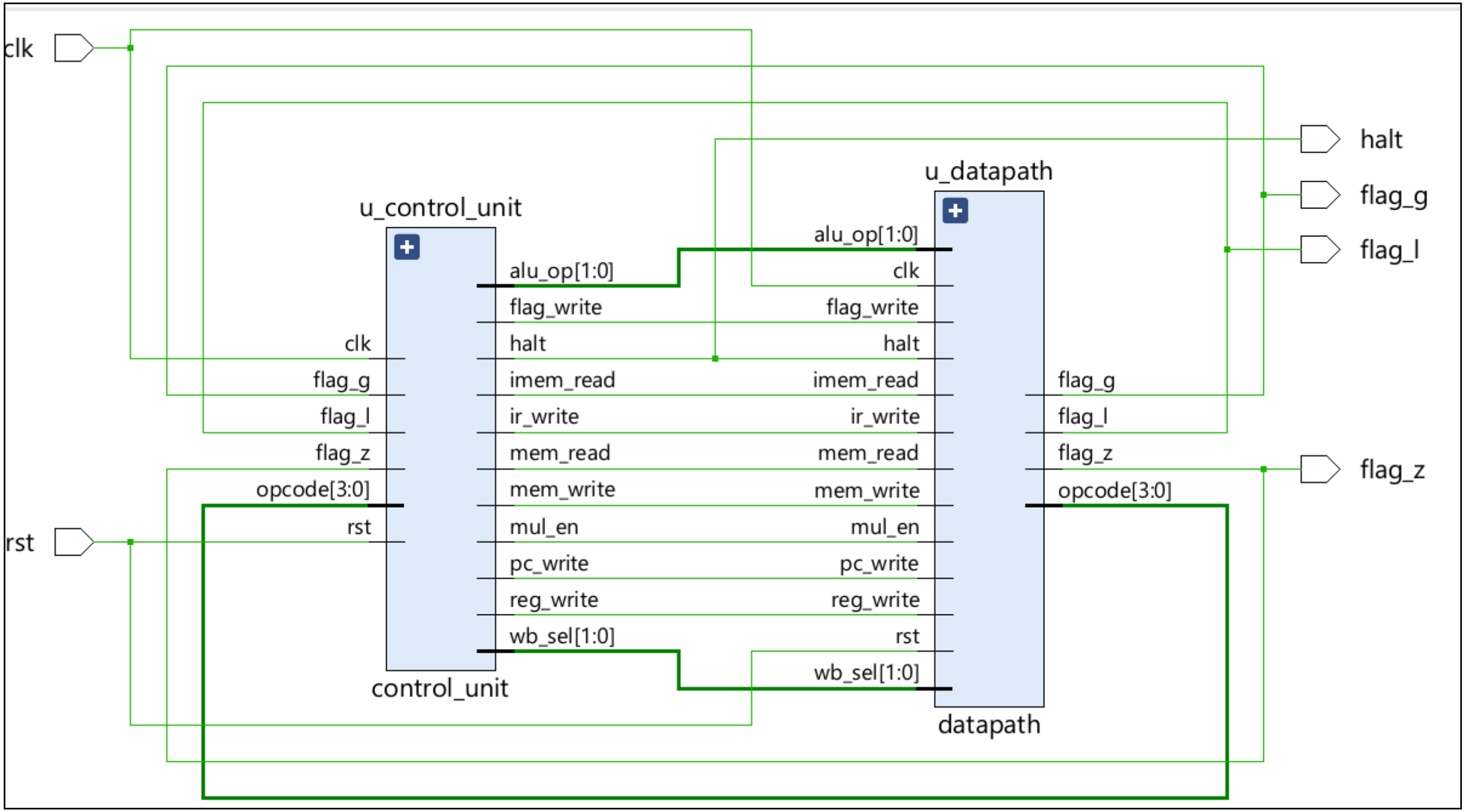
```
620A
640B
1650
2850
3AC0
7A14
41C0
F000
```

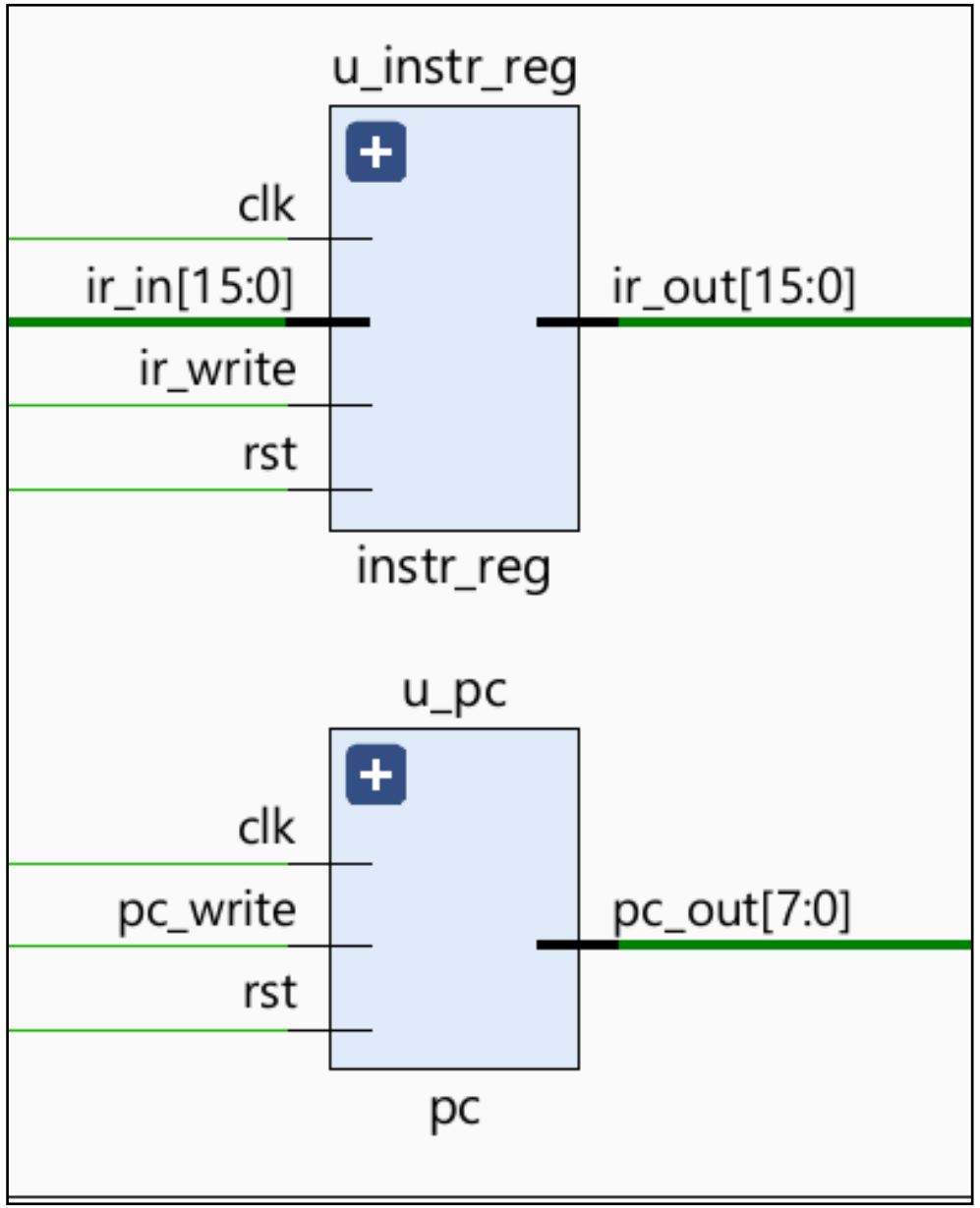
program2.mem

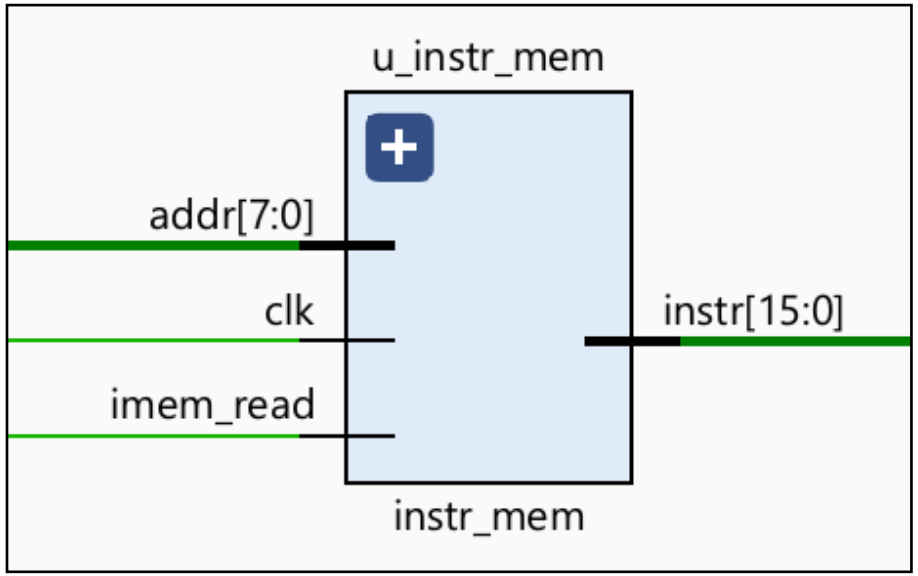
```
620A
640B
1650
3850
7815
F000
```

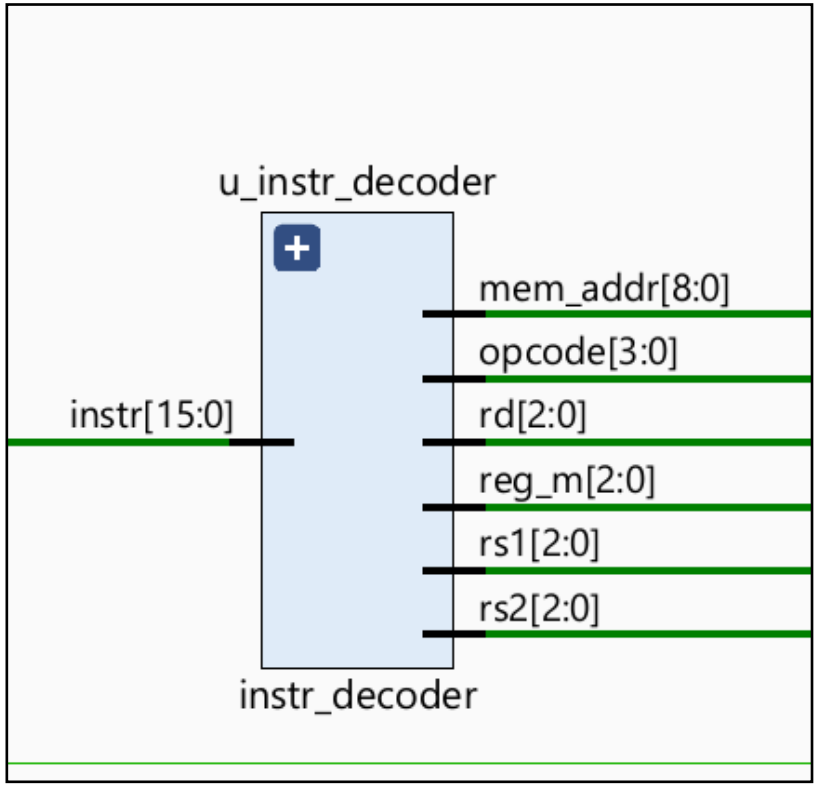
Problem statement

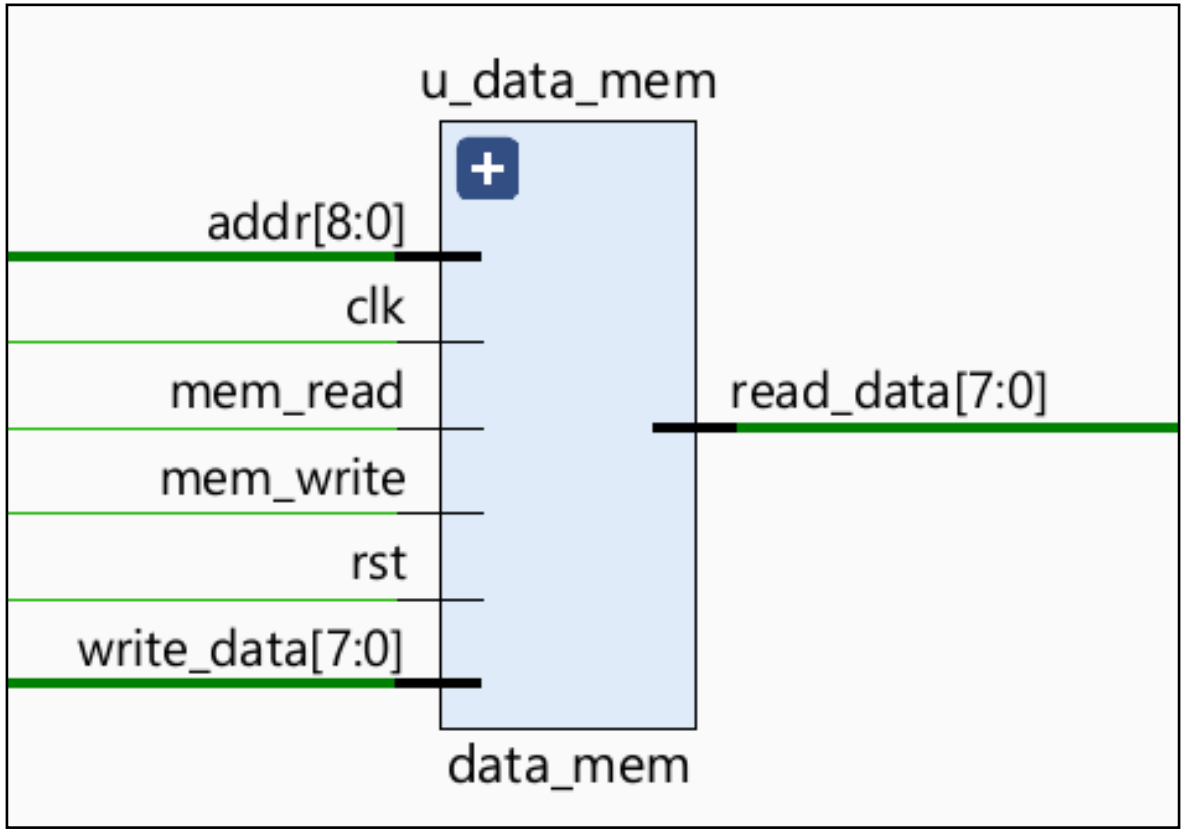
- Design an instruction memory (ROM) for the RISC Processor.
- Load **program1 or program2** into the memory using VIVADO
- Run the consecutive read operations (in every clock cycle) and show the instructions one by one on the seven-segment display.

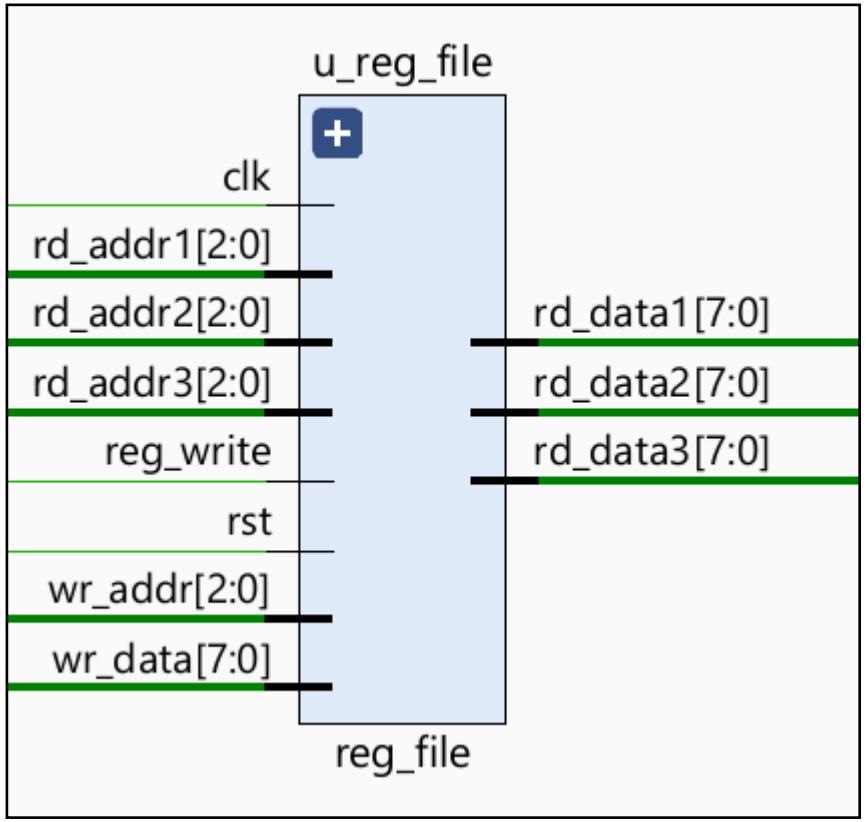

clk
rst
u_control_unit
clk
flag_g
flag_l
flag_z
opcode[3:0]
rst
alu_op[1:0]
flag_write
halt
imem_read
ir_write
mem_read
mem_write
mul_en
pc_write
reg_write
wb_sel[1:0]
control_unit
u_datapath
alu_op[1:0]
clk
flag_write
halt
imem_read
ir_write
mem_read
mem_write
mul_en
pc_write
reg_write
rst
wb_sel[1:0]
flag_g
flag_l
flag_z
opcode[3:0]
datapath
halt
flag_g
flag_l
flag_z

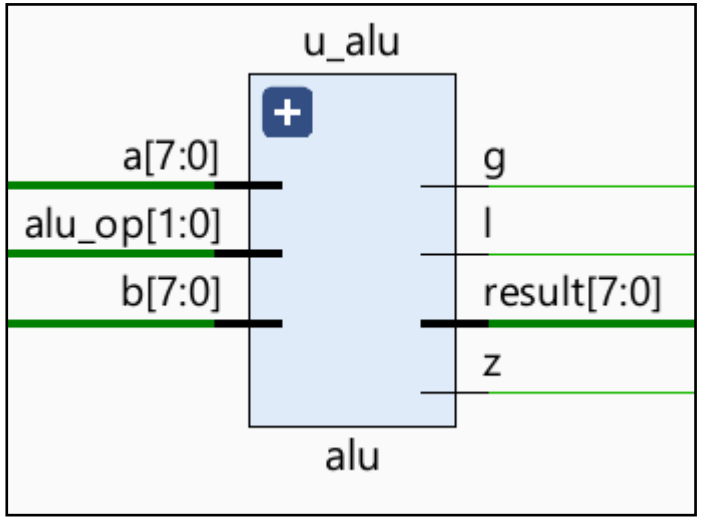

u_instr_reg
clk
ir_in[15:0]
ir_write
rst
ir_out[15:0]
instr_reg
u_pc
clk
pc_write
rst
pc_out[7:0]
pc

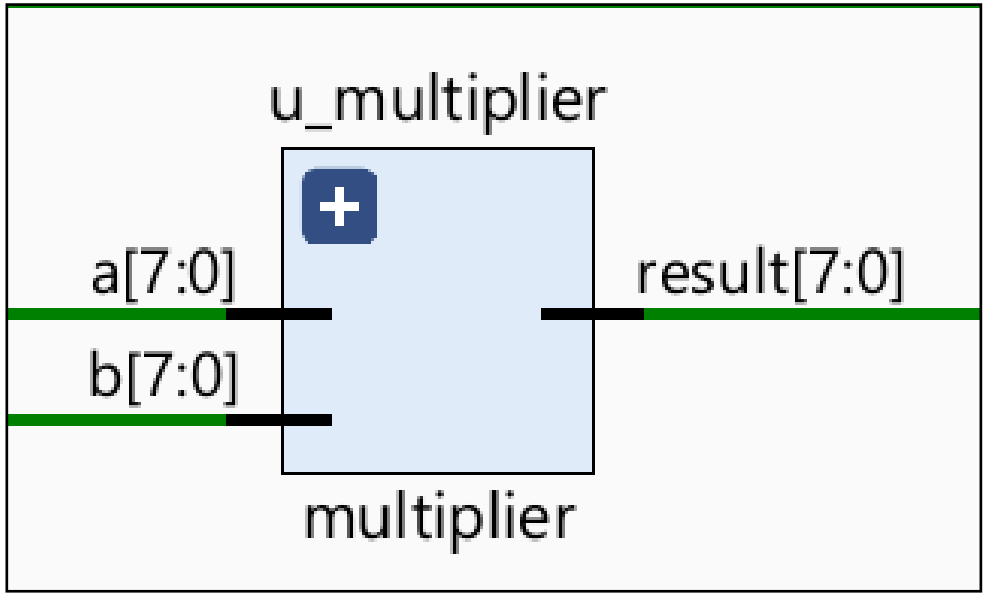

u_instr_mem
addr[7:0]
clk
imem_read
instr[15:0]
instr_mem

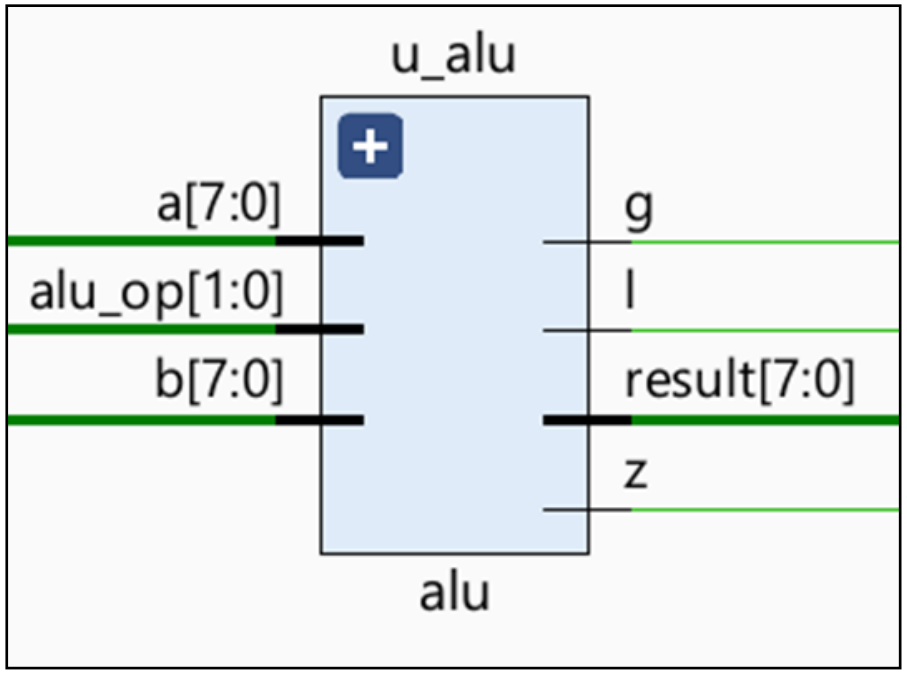

u_instr_decoder
instr[15:0]
mem_addr[8:0]
opcode[3:0]
rd[2:0]
reg_m[2:0]
rs1[2:0]
rs2[2:0]
instr_decoder


u_data_mem
addr[8:0]
clk
mem_read
mem_write
rst
write_data[7:0]
read_data[7:0]
data_mem


u_reg_file
clk
rd_addr1[2:0]
rd_addr2[2:0]
rd_addr3[2:0]
reg_write
rst
wr_addr[2:0]
wr_data[7:0]
rd_data1[7:0]
rd_data2[7:0]
rd_data3[7:0]
reg_file


u_alu
a[7:0]
alu_op[1:0]
b[7:0]
g
l
result[7:0]
z
alu


u_multiplier
a[7:0]
b[7:0]
result[7:0]
multiplier


u_alu
a[7:0]
alu_op[1:0]
b[7:0]
g
l
result[7:0]
z
alu

```
module pc (
    input              clk,
    input              rst,
    input              pc_write,
    output reg [7:0]   pc_out
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            pc_out <= 8'd0;
        else if (pc_write)
            pc_out <= pc_out + 8'd1;
    end
endmodule
```

```
module instr_mem (
    input              clk,
    input              imem_read,
    input      [7:0]   addr,
    output reg [15:0]  instr
);
    reg [15:0] mem [0:255];

    initial begin
        $readmemh("program1.mem", mem);
    end

    always @(posedge clk) begin
        if (imem_read)
            instr <= mem[addr];
    end
endmodule
```

```
module instr_decoder (
    input       [15:0] instr,
    output      [3:0]  opcode,
    output      [2:0]  rd,
    output      [2:0]  rs1,
    output      [2:0]  rs2,
    output      [2:0]  reg_m,
    output      [8:0]  mem_addr
);
    assign opcode   = instr[15:12];
    assign rd       = instr[11:9];
    assign rs1      = instr[8:6];
    assign rs2      = instr[5:3];
    assign reg_m    = instr[11:9];
    assign mem_addr = instr[8:0];
endmodule
```